TP : entraîner un modèle par descente de gradient

Journée des Gymnasiens

Objectif

Comme nous l'avons vu dans l'introduction ce matin, l'apprentissage automatique consiste à chercher des *paramètres* (des nombres) qui rendent un *modèle* (une fonction) le plus proche possible des données. Cette recherche repose sur trois ingrédients que nous avons déjà rencontrés :

— un **modèle** : une fonction possédant des paramètres $p_0, p_1, \ldots$, que l'on ajuste pour qu'elle colle aux données ;

— des **données** : des exemples (x_i, y_i) que le modèle doit reproduire ;

— une fonction de **coût** (*loss*) : un score qui mesure l'écart entre les prédictions du modèle et les cibles y_i. Il doit être grand quand le modèle se trompe beaucoup et petit quand il colle bien aux données.

Et nous avons vu enfin comment trouver les paramètres automatiquement : la **descente de gradient**, qui à chaque étape modifie les paramètres dans le sens opposé au gradient du coût :

$$p \;\leftarrow\; p - \eta\, \nabla L(p)$$

où η est le *taux d'apprentissage*.

Dans ce TP, vous allez vivre cette expérience de l'intérieur, en programmant vous-même les trois briques. Le fichier `src/lab.py` ne contient que les **trois fonctions à compléter** : `model`, `loss` et `grad`. Pour l'instant elles ne font rien : elles ne renvoient que le symbole `...`. Tout le reste (données, vérifications automatiques, animation d'entraînement) est déjà écrit dans le lanceur `src/lab_run.py`, qui importe vos trois fonctions.

Après chaque modification, relancez le programme dans VS Code : appuyez sur `F5`. Le programme vérifie d'abord votre `model`, puis votre `loss`, puis votre `grad`, il lance ensuite l'animation d'entraînement : elle ne marchera qu'une fois les trois fonctions prêtes. En cas d'erreur, la vérification affiche la valeur attendue et la valeur obtenue, ainsi que la taille des tableaux.

Étape 1 : le modèle `model`

Notre modèle est un polynôme de degré 3, paramétré par une liste `parameters` $= [p_0, p_1, p_2, p_3]$:

$$f(x) \;=\; p_0 \;+\; p_1\, x \;+\; p_2\, x^2 \;+\; p_3\, x^3 \;=\; \sum_{k=0}^{3} p_k\, x^k.$$

Le coefficient à la position k multiplie donc x à la puissance k.

Q1. Sans calcul : si `xs` contient N valeurs, quelle est la taille du tableau renvoyé par `model(parameters, xs)` ?

Q2. Implémentez `model`. L'opérateur `**` calcule les puissances (`xs**2`, `xs**3`…), et une boucle `for k, p in enumerate(parameters)` itère sur chaque coefficient p_k avec son rang k : Il faut donc multiplier `p` par `xs**k`, et ensuite sommer les résultats.

Q3. Relancez le programme avec `F5` : si votre implémentation est correcte, le message « `Le modèle a l'air bon.` » s'affichera. La vérification teste plusieurs valeurs, y compris un polynôme de degré plus élevé et plusieurs valeurs de x à la fois : votre implémentation doit aussi rester valable quand le degré du polynôme ou le nombre de points est différent.

Étape 2 : le coût `loss`

Comme vu ce matin, nous notons L le *coût* : le score du modèle pour un jeu de paramètres. Ici (comme dans la plupart des modèles à sortie numérique), nous prenons la moyenne des carrés des erreurs :

$$L(p) \;=\; \frac{1}{N}\Big(\big(f_p(x_1)-y_1\big)^2+\cdots+\big(f_p(x_N)-y_N\big)^2\Big) \;=\; \frac{1}{N}\sum_{i=1}^{N}\big(f_p(x_i)-y_i\big)^2.$$

Q4. Que vaut L si le modèle colle parfaitement aux données ? Et si le modèle ne donne pas la bonne valeur au moindre point des données, que peut-on dire de L ?

Q5. Implémentez `loss(parameters, xs, ys)` : la moyenne des carrés des différences entre les prédictions de votre `model` et les cibles `ys`. Comme elle fait appel à votre `model`, elle ne peut fonctionner qu'une fois l'étape 1 terminée. La boucle `for x, y in zip(xs, ys):` itère sur les paires (valeur de x, cible y) du jeu de données ; il faut ensuite sommer les carrés des différences, puis diviser par la taille du jeu de données.

Q6. Relancez le programme avec `F5` : si vous ne vous êtes pas trompé·e, le message « `Le coût a l'air bon.` » s'affichera.

Étape 3 : le gradient `grad`

Nous avions vu ce matin que la descente de gradient demande de connaître la dérivée partielle $\dfrac{\partial L}{\partial p_i}$ pour chaque paramètre p_i : de combien L varie si l'on bouge p_i d'une toute petite quantité. C'est sans doute la partie la plus mathématique du TP, mais nous allons la calculer *élément par élément* : la collection de ces dérivées est le **gradient** ∇L, il a une entrée par paramètre, et son i-ème élément est la dérivée de L par rapport à p_i.

Q7. *Sur papier : les résultats sont donnés, à vous de comprendre pourquoi.* Pour un point de données (x, y), posons $u = f_p(x) - y$ (l'erreur du point). La règle de dérivation des composées donne $\big(u^2\big)' = 2\,u\,u'$:

(a) Montrez que

$$\frac{\partial\big(u^2\big)}{\partial p_i} \;=\; 2\,u \times \frac{\partial f_p(x)}{\partial p_i}\;:$$

pourquoi la cible y disparaît de la dérivée ?

(b) Prouvez que

$$\frac{\partial f_p(x)}{\partial p_i} \;=\; x^i\;:$$

(dans $f_p(x) = p_0 + p_1\,x + p_2\,x^2 + \cdots$, seul le terme en p_i dépend de p_i.)

(c) En déduire, en moyennant sur les N points de données :

$$\frac{\partial L}{\partial p_i} \;=\; \frac{2}{N}\sum_{j=1}^{N}\big(f_p(x_j)-y_j\big)\,x_j^i\,.$$

Q8. Concrètement dans le code : le i-ème élément du gradient, à savoir $\dfrac{\partial L}{\partial p_i}$, vaut la moyenne sur les données de $2\big(f_p(x_j)-y_j\big)\,x_j^i$. Implémentez `grad(parameters, xs, ys)` : la boucle `for k in range(len(parameters)):` itère les rangs i, et comme tout à l'heure `for x, y in zip(xs, ys):` itère sur les paires (valeur de x, cible y) du jeu de données. Pour chaque rang k, il faut ensuite sommer les éléments, puis diviser par la taille du jeu de données.

Et maintenant : entraînez !

Maintenant que `model`, `loss` et `grad` existent, le programme *s'entraîne* tout seul : partant des paramètres $p = (0,0,0,0)$, il effectue 250 itérations de $p \leftarrow p - \eta\,\nabla L(p)$ et affiche l'animation d'entraînement (à gauche les données et les prédictions du modèle, à droite la courbe du coût).

Faites tourner le programme avec `F5` et observez : le polynôme approxime les points de mieux en mieux. **Vous avez entraîné votre premier modèle ! Félicitation !**